



Verdict

Track 1 remains the best choice for you, but the previous GPT conversation is directionally right—not fully proven. Its strongest insight is to avoid building another chatbot or generic payment firewall; its weakest point is treating AP2 compatibility, competitor differentiation, and the “89.5/100” score as more certain than the available evidence allows.

Your strongest project direction is:

RupeeProof: a transaction-integrity gateway that converts AI purchase intent into a verifiable, bounded Razorpay payment action.

This fits Razorpay’s official Track 1 requirement: either grow merchant revenue using Razorpay test-mode APIs or make a merchant transactable by an AI buyer end to end, while keeping money actions explainable, bounded, gated, auditable, and capable of graceful failure. ^[1] ^[2]

What the GPT conversation got right

Track 1 selection

The recommendation to choose **AI Growth & Agentic Commerce** is sound because it aligns directly with Razorpay’s current product direction. Razorpay publicly describes Agentic Payments as supporting AI-native payments across apps, LLMs, and voice, while its Agent Studio focuses on payment operations and commerce workflows. ^[2] ^[3]

Track 1 also gives you the best combination of:

- Direct Razorpay relevance.
- A visually clear live demo.
- Real test-mode API integration.
- A strong engineering story.
- A problem that can be evaluated without private production data.

The previous GPT was also correct that Track 3 is dangerous if approached as a normal failed-payment or abandoned-cart bot. Razorpay already publicly lists agents for abandoned-cart conversion and subscription recovery, so a basic recovery agent would look like a product clone. ^[3] ^[4]

Separating AI from authority

This is probably the best technical decision in the conversation:

“The model interprets intent. It never grants authority.”

That boundary is appropriate for a payments system. An LLM can extract constraints from language, identify ambiguity, classify an action, and explain a decision. It should not independently decide whether a payment is authorized.

The deterministic layer should own:

- Amount ceilings.
- Merchant binding.
- Cart binding.
- Expiry.
- Nonce and replay protection.
- Idempotency.
- Approval requirements.
- Action permissions.
- Razorpay execution.

This architecture gives you a credible answer to the judge’s likely question: **“What happens when the model is wrong?”**

Focusing on failure cases

The proposed demo—valid action, amount mutation, merchant substitution, replay, stale cart, and duplicate request—is strong because the failure is observable. Judges can see the system reject a dangerous action before it reaches Razorpay.

This is better than showing a vague claim such as “our AI improves payment security.” A concrete denial reason like `AMOUNT_OUTSIDE_MANDATE` is easier to understand, test, and defend.

Using Razorpay as execution, not decoration

The previous conversation correctly insisted on one real Razorpay test-mode path. Razorpay’s official MCP documentation describes more than 35 tools mapped to payment operations such as payments, payment links, orders, refunds, QR codes, settlements, and payouts. ^[5]

That creates a meaningful technical opportunity: your gateway can sit between an AI agent and a real payment-operation surface rather than merely displaying simulated transactions.

What the GPT conversation got wrong or overstated

The “89.5/100” score is not evidence

The score comparing Track 1 with Tracks 2–5 is a useful internal decision framework, but it is not an official Razorpay score and cannot prove that Track 1 has an 89.5% chance of success. The attached report itself describes the numbers as a weighted matrix, meaning they are judgments based on assumptions rather than measured competition data. [1]

Use the score as a prioritization tool, not as proof.

A more honest conclusion is:

Claim	Status
Track 1 aligns strongly with Razorpay’s published brief	Strongly supported
Track 1 has a visually compelling demo	Reasonable design judgment
Track 1 is less crowded than recovery or reconciliation	Directional, not proven
RupeeProof will beat other submissions	Unknown
Track 1 deserves exactly 89.5/100	Subjective
AP2-to-Razorpay execution is a genuine gap	Plausible, but must be demonstrated

“AP2 already specifies everything” is only partially true

The previous GPT was correct that AP2 defines signed authorization concepts. Google describes AP2 mandates as cryptographically signed proof of user instructions, including rules such as price limits and timing. [6]

The official AP2 documentation also distinguishes checkout and payment mandates. [7] [8]

However, this does not prove that AP2 completely solves your proposed product. AP2 defines a protocol layer; it does not automatically provide:

- A Razorpay-specific adapter.
- Razorpay test-mode order execution.
- Merchant policy management.
- Cart-state reconciliation.
- Local replay protection.
- Razorpay webhook processing.
- A user-friendly approval UI.
- A conformance harness for Razorpay operations.
- Operational evidence for merchants.

Therefore, the GPT’s strategic correction is right, but the wording should change from:

“AP2 already specifies much of this, so we must use AP2.”

to:

“AP2 provides an authorization reference model; our project demonstrates how equivalent authorization evidence can be verified and executed safely against Razorpay.”

Do not claim full AP2 compatibility unless your implementation follows the actual AP2 schemas, credential types, signature model, and verification rules.

“AP2 is the missing bridge” needs narrower wording

The statement that “AP2 defines authorization, Razorpay exposes execution, and RupeeProof connects them” is a compelling product narrative, but it should be presented as your architectural thesis—not as an officially recognized industry gap.

The AP2 ecosystem is designed to work alongside other agentic protocols, while ACP focuses on connecting buyers, agents, and businesses for checkout.^{[9] [10]} That means your differentiated contribution should be explicitly Razorpay-native:

“RupeeProof is a Razorpay execution and evidence layer inspired by AP2-style authorization, with transaction-state verification before money operations.”

If you implement a real subset of AP2, say exactly which subset. If you only use AP2 concepts, call it AP2-inspired rather than AP2-compliant.

The SENTINEL threat is not independently verified

The attached report describes a public SENTINEL repository with MCP interception, policy enforcement, audit logs, and Razorpay test-mode evidence.^[1] However, the internet research performed here did not independently verify the repository’s claimed test count, architecture, or actual Razorpay behavior.

Treat SENTINEL as a credible adjacent competitor identified in your research, not as a proven benchmark.

More importantly, the previous GPT’s positioning is still strategically useful:

SENTINEL-like system	RupeeProof
Controls whether an agent may call a tool	Proves whether a specific payment action matches authorization
Generic MCP policy boundary	Razorpay transaction-state boundary
Tool name and argument policy	Intent, merchant, cart, amount, expiry, nonce, state
“Can this agent call refund?”	“Can this exact refund be executed under this authorization?”
Security proxy	Payment authorization and execution evidence

Do not claim that the two products are completely different. They overlap around policy enforcement. Your differentiation must come from signed state binding and payment

evidence, not from the words “firewall,” “agent security,” or “MCP guard.”

Competitor-count claims are too strong

The attached report cites a public scan of 140 repositories and summarizes many competitor projects.^[1] That is useful intelligence, but it does not establish that those repositories are official applicants, active competitors, complete projects, or high-quality submissions.

The internet search did find at least one public Track 1-style project, RazorGrowth Agent, which describes natural-language product selection, a deterministic policy gate, spending limits, idempotency, human approval, an agent-readable catalog, and Razorpay execution.^[11]

This is important: **your original RupeeFence concept is already close to at least one publicly described build.** A generic “LLM → policy gate → Razorpay” flow is therefore not enough.

Your advantage must be the deeper proof layer:

- Signed authorization object.
- Canonical cart hash.
- Merchant and catalog version binding.
- Re-approval after state change.
- Replay resistance.
- Webhook-linked execution receipt.
- Public adversarial evaluation.
- Comparison against a raw-agent baseline and a simple static policy baseline.

“100% blocked” claims require careful definitions

The previous conversation proposed statements such as blocking all known mutations across 10,000 traces. That can be a good result, but only if you publish:

- The attack-generation rules.
- The expected-decision oracle.
- The train/test or generation split.
- The benign cases.
- The exact meaning of “blocked.”
- Whether the test uses real Razorpay calls or a local simulator.
- Whether the test covers only your own known attack templates.

Do not say:

“RupeeProof prevents payment fraud.”

Say:

“RupeeProof blocked all policy violations in our seeded test corpus under the defined oracle, while the Razorpay integration was validated in test mode.”

That wording is technically defensible.

The best version of the project

Project definition

Name: RupeeProof

Track: AI Growth & Agentic Commerce

Category: Verifiable agent transaction execution

One-line pitch:

RupeeProof lets an AI agent propose a payment, but only executes it when the exact merchant, cart, amount, state, and authorization can be proven.

The user journey

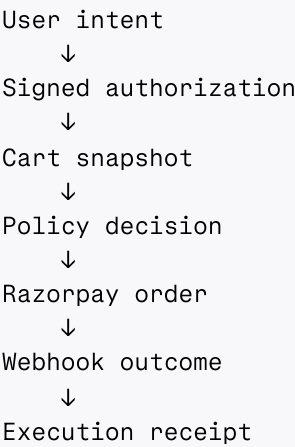
1. The user says: “Buy these headphones from Merchant A for less than ₹2,000.”
2. The LLM extracts structured constraints.
3. The system asks for clarification if the request is ambiguous.
4. A signed authorization object is created.
5. The agent proposes a cart.
6. The cart is canonicalized and hashed.
7. The deterministic gateway checks the authorization against the cart.
8. The gateway creates a Razorpay test-mode order or payment link.
9. Webhooks are verified and associated with the transaction.
10. The system produces a receipt showing what was authorized, what was executed, and why.

The hero demo

Use one transaction and four mutations:

Demo step	Expected result
₹1,899, correct merchant, unchanged cart	Allowed; Razorpay test-mode order created
Amount changed to ₹2,499	Denied: amount exceeds mandate
Merchant changed from A to B	Denied: merchant mismatch
Original request replayed	Denied: mandate already consumed
Price or shipping changed after approval	Re-approval required
Webhook delivered twice	Second event deduplicated

The strongest visual is not a dashboard with many charts. It is a **transaction evidence timeline**:



What should be removed from the current plan

Cut these unless the core flow is already stable:

- Voice interface.
- Multi-agent orchestration.
- Marketplace functionality.
- Generic product search.
- Custom foundation model.
- Full AP2 ecosystem implementation.
- Large analytics dashboard.
- Live production payment movement.
- Broad MCP tool coverage.
- Claims of preventing real financial loss.

The official page asks for a working build, public repository, five-minute pitch video, architecture, and an explanation of what broke and how it was fixed. ^[1] That favors a small, reproducible, deeply tested system over a broad but fragile platform.

What should be added

A claim ledger

Create a file called CLAIMS.md with columns like:

Claim	Evidence	Status
Razorpay order creation works	Redacted test-mode request and response	Real test mode
Amount mutation is denied	Automated test trace	Reproduced
AP2 terminology is used correctly	Official AP2 documentation	Externally sourced

Claim	Evidence	Status
10,000 traces were evaluated	Committed result artifact	Reproducible
No unsafe call reaches Razorpay	Gateway logs and test fixtures	Demonstrated within test scope

This will immediately make the repository look more mature.

A baseline comparison

Compare three systems:

- **B0:** Raw agent directly calls the payment adapter.
- **B1:** Static tool and amount allow-list.
- **B2:** RupeeProof with signed authorization, cart hash, state, nonce, and idempotency.

Measure:

- Unsafe-forward rate.
- Valid-action pass rate.
- Attack recall by class.
- False-denial rate.
- Audit completeness.
- p95 decision latency.
- Number of actions requiring approval.

This proves that RupeeProof is more than a visually attractive policy screen.

A merchant-growth connection

The main risk is that judges may see RupeeProof as security plumbing rather than growth. Connect trust directly to business value:

Merchants will be more willing to expose products and payment actions to AI buyers when each action is bounded, verifiable, and auditable.

Do not claim that you increased GMV unless you actually run a controlled experiment. Instead, show the prerequisite for agentic commerce: **safe transaction execution with evidence.**

Track decision matrix

Track	Recommendation	Reason
Track 1: Agentic Commerce	Choose this	Best match for your architecture, Razorpay's public direction, and your strongest demo
Track 2: Risk Manager	Avoid unless you have strong labeled data	Requires honest held-out precision/recall and false-positive economics; generic fraud is crowded

Track	Recommendation	Reason
Track 3: Revenue Recovery	Backup option	Easy to understand but crowded by Razorpay's own Agent Studio and public recovery projects
Track 4: Finance Controller	Feasible backup	Highly measurable, but reconciliation is already a Razorpay product area and has many similar projects
Open Track	Avoid for this submission	Maximum freedom but weaker Razorpay alignment and more burden to establish the problem

Final recommendation

Build Track 1, but change the positioning from:

“A secure AI shopping/payment firewall.”

to:

“A Razorpay-native proof layer for AI-initiated payments.”

The previous GPT conversation is right about the direction, architecture, demo shape, and need to avoid generic agent security. It is wrong to present the subjective score, competitor landscape, AP2 gap, and benchmark claims as proven facts.

The winning version is not “we invented a better mandate.” It is:

“We took the authorization problem seriously, bound it to a real cart and Razorpay execution state, and built a reproducible proof that the agent paid only what the user authorized.”

One final correction: the official page confirms that applications close on **September 5**, but the page content available here does not establish a precise cutoff time or timezone.^[1] Treat **September 4 evening IST** as your internal deadline and submit once the smallest credible flow, public repository, architecture, and five-minute video are ready.

✱

1. [razorpay-buildathon-research.html](#)
2. <https://razorpay.com/agentic-payments/>
3. <https://razorpay.com/newsroom/razorpay-launches-the-worlds-first-ai-native-agent-studio-for-payments-at-ftx26-powered-by-anthropics-claude/>
4. <https://fintechnews.sg/127652/fintech-india/razorpay-agent-studio-ai-payments-claude/>
5. <https://razorpay.com/docs/mcp-server/tools-reference/>
6. <https://cloud.google.com/blog/products/ai-machine-learning/announcing-agents-to-payments-ap2-protocol>
7. https://ap2-protocol.org/ap2/payment_mandate/
8. https://ap2-protocol.org/ap2/checkout_mandate/
9. <https://agentmint.net/agentic-payment-protocols/>
10. <https://github.com/agentic-commerce-protocol/agentic-commerce-protocol>

11. https://www.linkedin.com/posts/saniya-mihani-4a5742223_razorpaybuildathon-agenticcommerce-aiagents-activity-7496785730883649536-DJdZ
12. <https://razorpay.com/buildathon/>
13. <https://cloudsutra.in/blogs/razorpay-ai-buildathon-2026-75000-stipend-internship-for-students>
14. <https://velonx.in/blog/razorpay-ai-buildathon-2026-tracks-eligibility-stipend-selection-process>
15. <https://x.com/ranaharshraj7/status/2090467817900687515>
16. <https://fresherjobinfo.in/razorpay-ai-buildathon-internship-2026/>
17. <https://www.placement-officer.com/2026/08/razorpay-ai-buildathon-2026-build-ai.html>
18. https://x.com/suraj_sharma14/status/2091027586130587919
19. https://www.linkedin.com/posts/ashinee20_razorpay-ai-buildathon-2026-build-show-activity-7496914271059144704-ukJE
20. <https://dev.to/devengers/dev-opportunity-radar-14-mlh-global-hack-week-40k-agents-for-humans-hackathon-razorpay-ai-2b9g>
21. <https://razorpay.com/sprint/26>
22. <https://www.investing.com/news/cryptocurrency-news/the-agentic-economy-takes-center-stage-at-sui-basecamp-2026-in-singapore-4879557>
23. <https://www.mk.co.kr/en/it/12135767>
24. <https://www.fintechbiznews.com/fintech-digital-payment/rategain-razorpay-partner>
25. <https://www.techcircle.in/2026/03/12/razorpay-rolls-out-ai-agent-studio-to-automate-payments-for-businesses/>
26. <https://www.moneycontrol.com/news/business/startup/razorpay-rolls-out-ai-agent-studio-to-automate-payment-ops-built-using-anthropic-s-claude-13858737.html>
27. <https://www.aninews.in/news/business/razorpay-unveils-worlds-first-agent-studio-to-automate-payments-launches-a-gentic-experience-platform20260312114433>
28. <https://agentspay.ai/blog/payment-mandates>
29. <https://www.thehindubusinessline.com/money-and-banking/razorpay-launches-worlds-first-ai-agent-studio-for-payments/article70734004.ece>
30. <https://theheadandtale.com/fintech-news/razorpay-rolls-out-b2b-agent-marketplace/>
31. <https://razorpay.com/docs/mcp-server/integrations/?preferred-country=US>
32. <https://agentspay.ai/vs/ap2>
33. https://www.linkedin.com/posts/the-skillians_razorpay-ai-buildathon-came-across-this-activity-7498411969125994496-dzqQ
34. https://www.linkedin.com/posts/schandakeshroshan_razorpay-ai-buildathon-build-show-get-activity-7496419284542423040-hqSD
35. <https://github.com/razorpay>
36. https://www.linkedin.com/posts/g-c-r-g-group-of-institutions_razorpay-razorpayinternship-aibuilder-activity-7497697239134547968-mKjA
37. https://www.linkedin.com/posts/iambibinbabu_openais-walmart-agent-got-12-merchants-activity-7457390960532959232-6LDj
38. https://www.linkedin.com/posts/razorpay_claude-x-razorpay-activity-7439221461778837504-ZTJg
39. https://www.linkedin.com/posts/razorpay_razorpayftx26-activity-7439292421664038912-w8fL

40. https://www.linkedin.com/posts/razorpay_razorpayftx26-activity-7438232356701868032-JPTg
41. https://www.linkedin.com/posts/shubham-patil-03019a283_reviveai-autonomous-multi-agent-revenue-activity-7497922218778329089-VbHE
42. https://www.linkedin.com/posts/ashutoshpsingh_inappagents-appagents-aiagents-activity-7438988068902129664-haOe
43. https://www.linkedin.com/posts/razorpay_ai-at-razorpay-shaping-the-company-for-the-activity-7439567417824997376-oLHE
44. https://www.linkedin.com/posts/razorpay_vishal-dhupar-activity-7437833905023614976-h0f9